SAMPLE ANSWER · HIGH-PROBABILITY QUESTION

Design a URL Shortener

A complete interview answer: requirements, key entities, API design, data modelling, architecture, optimisation, trade-offs, and one-line follow-ups.

P0 very high | 3–5 YOE | Read-heavy · 100:1 | 3 diagrams | 9 pages

WHY INTERVIEWERS ASK IT

One question, many fundamentals

- Unique ID generation and collision handling
- Key-value modelling and secondary indexes
- Cache-aside and hot links
- Read-heavy scaling and replication
- Async analytics and failure handling

WHAT STRONG CANDIDATES DO

Make the read path obvious

- Scope the product before drawing boxes
- Define entities and keys before the architecture
- Draw one clean diagram, not five sketches
- Explain 301 vs 302 and cache behaviour
- Move analytics off the redirect path

60-SECOND ANSWER

"A URL shortener is a read-heavy key-value system. The write path validates the URL, generates a unique short code, stores `code → long URL` in a sharded, replicated KV store, and caches it. The redirect path reads the cache, falls back to the KV store on a miss, returns a 301 or 302, and publishes the click event asynchronously. I'd keep app servers stateless, shard by short code, and use a queue for analytics so the redirect stays fast."

Answer map

Requirements → Key entities → API → Data model → Architecture → Optimise → Follow-ups

02 Requirements & estimates
03 Key entities & API design
04 Data modelling & code generation
05 Architecture diagram
06 Write flow
07 Redirect flow
08 Optimisation & trade-offs
09 Follow-ups, one-liners, traps

1. Requirements

FUNCTIONAL

What it must do

- Create a short link from a long URL.
- Redirect `GET /{code}` to the original URL.
- Optional custom alias and expiry.
- Optional click analytics.
- Disable or delete a link.

NON-FUNCTIONAL

How well

- Redirect p99 under ~100–200 ms.
- Very high availability; redirects must not depend on analytics.
- Read-heavy: ~100 redirects per write.
- Short codes unique and hard to guess if security matters.
- Analytics can be eventually consistent.

Assumptions and estimates

- 100 million new links per day; 10 billion redirects per day.
- Peak traffic is 2–3× average; average mapping is ~500 bytes.
- Over five years: ~183 billion links, ~91 TB raw, ~300 TB with replication/index overhead.

WRITES / SEC
~1.2k
avg · ~3.5k peak

REDIRECTS / SEC
~116k
avg · ~350k peak

LINKS OVER 5 YEARS
~183B
100M × 365 × 5

STORAGE WITH REPLICATION
~300 TB
raw + copies + indexes

PRESSURE	CONSEQUENCE
~350k redirects/sec at peak	Stateless app tier, cache, many KV shards
~3.5k writes/sec at peak	ID generation and unique-code check
~300 TB over five years	Shard the KV store; do not use one node
Every redirect produces analytics	Queue + workers; never block the redirect

ESTIMATION RULE

The exact numbers are assumptions. What matters is the ratio: reads dominate, so the redirect path gets the cache and the simplest lookup. Analytics and other slow work stay off that path.

2. Key entities

CORE ENTITIES

Three entities are enough

- **User** — optional owner of a link.
- **UrlMapping** — short code, long URL, owner, expiry, status.
- **ClickEvent** — append-only event for analytics.

RELATIONSHIPS

Simple ownership

- User 1 → N UrlMapping.
- UrlMapping 1 → N ClickEvent.
- A mapping is uniquely identified by `code`.
- Click events are never queried during redirect.

3. API design

OPERATION	CONTRACT	NOTES
Create	`POST /urls` `{ longUrl, customAlias?, expiresAt? }`	201 with short URL; 400 invalid; 409 alias conflict; optional `Idempotency-Key`.
Redirect	`GET /{code}`	301 or 302 with `Location`; 404 unknown; 410 expired.
Delete	`DELETE /urls/{code}`	204; soft delete; invalidate cache.
Stats	`GET /urls/{code}/stats`	Reads analytics store; never on redirect path.

CREATE RESPONSE

Example

`POST /urls`
`{ "longUrl": "https://example.com/very/long" }`

`201 Created`
`{ "code": "aB3xK9p", "shortUrl": "https://sho.rt/aB3xK9p" }`

REDIRECT CONTRACT

Keep it cheap

`GET /aB3xK9p`

`302 Found`
`Location: https://example.com/very/long`

No body required; cache headers only if the redirect is permanent.

API DECISIONS WORTH STATING

Use an `Idempotency-Key` on create to survive client retries. Use 302 by default if you need analytics or revocable links. Add `Cache-Control` only when the redirect is truly permanent; otherwise a CDN/browser may keep sending users to the old destination.

4. Data modelling

WRITE MODEL

`urls`

- `code` — primary key, unique
- `long_url` — normalized URL
- `owner_id` — optional
- `created_at`, `expires_at`
- `status` — active / disabled / deleted

ANALYTICS MODEL

`click_events`

- `event_id`, `code`, `ts`
- country / device / referrer (optional)
- Append-only; partition by time
- Aggregated by workers
- Never read on redirect

MODELLING DECISION	CHOICE	WHY
Primary lookup	`code` → `long_url`	One O(1) key lookup on the redirect path
Sharding	Hash of `code`	Even distribution; single-shard redirect lookup
Secondary index	`(owner_id, created_at)`	List a user's links without scanning
Duplicate prevention	Unique `(owner_id, long_url)`	Makes create idempotent for the same owner
Expiry	`expires_at` + lazy check + cleanup job	No extra lookup on redirect; reclaim space later
Cache	`code → long_url`, TTL	Hot links stay fast; negative cache for missing codes

Short-code generation

APPROACH	HOW	TRADE-OFF
Counter + base62	Increment a segment counter; encode in base62.	Short, no collisions; needs an ID service or preallocated ranges.
Hash + collision check	Hash URL, take N chars, retry on collision.	No ID service; collisions and extra writes.
Random code	7 random base62 chars; unique constraint.	Hard to guess; collision probability grows with volume.

DEFAULT

A 7-character base62 code gives roughly **$62^7 \approx 3.5$ trillion** combinations. Use a counter with preallocated ranges when you want no collisions; use random codes when guessability and central coordination matter. Always enforce uniqueness with a conditional write.

5. Architecture diagram

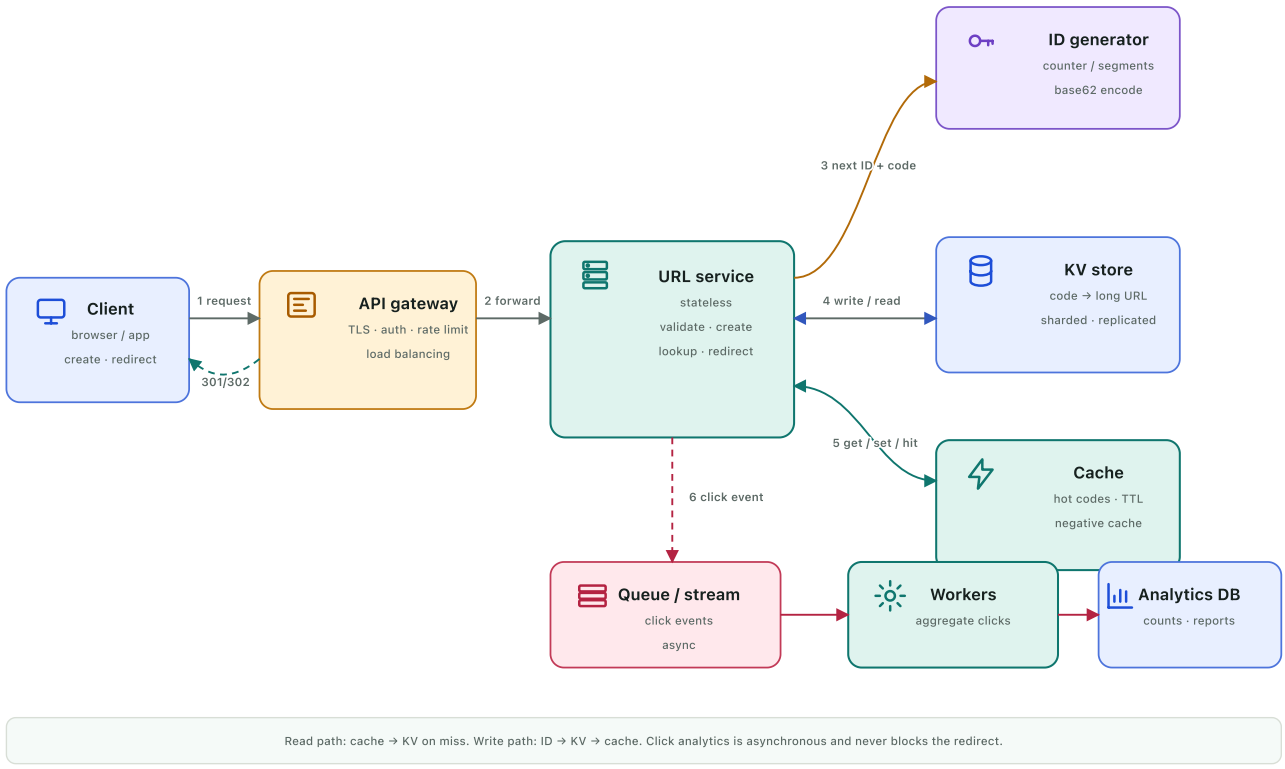


Figure 1 — The architecture. Icons mark clients, gateway, service, ID generation, storage, cache, queue, workers, and analytics.

CRITICAL PATH

Only what the redirect needs

- **Gateway:** TLS, auth, rate limit, routing.
- **URL service:** stateless; scales horizontally.
- **KV store:** O(1) lookup by code.
- **Cache:** absorbs hot-link traffic.

OFF THE CRITICAL PATH

Async by design

- **ID generator:** only on create.
- **Queue + workers:** analytics.
- **Analytics DB:** counters and reports.
- Analytics lag does not break redirects.

6. Write flow: create a short link

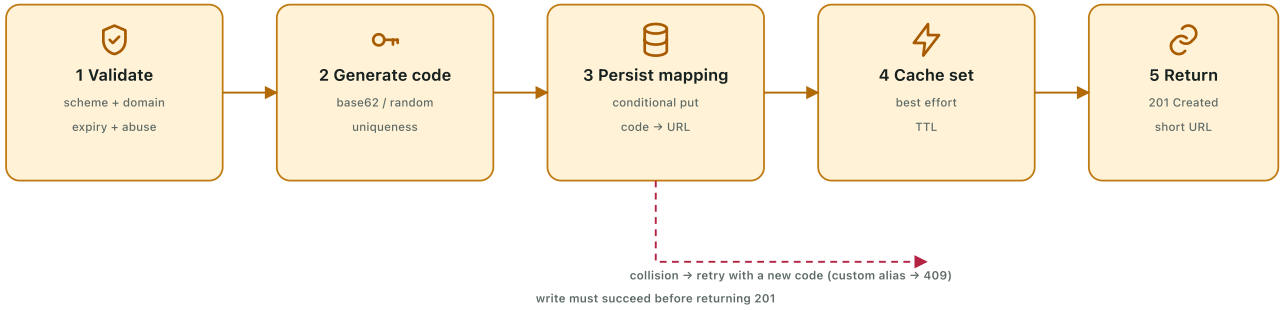


Figure 2 — Create flow. A successful write is the source of truth; cache population is best-effort.

What can go wrong

- **Collision:** retry with a new code; custom alias returns 409.
- **Duplicate create:** idempotency key or unique `(owner_id, long_url)`.
- **ID service down:** preallocate ranges per app instance or fall back to random codes.
- **Partial failure:** persist before returning success; never rely on cache as the source of truth.
- **Malicious URL:** validate scheme/domain, use safe-browsing/allowlist, rate-limit creates.

> **WHAT TO SAY**
>
> “I generate a unique code, write it with a conditional put, and only then return success. Cache population is an optimization, not the source of truth.”

7. Redirect flow: the hot path

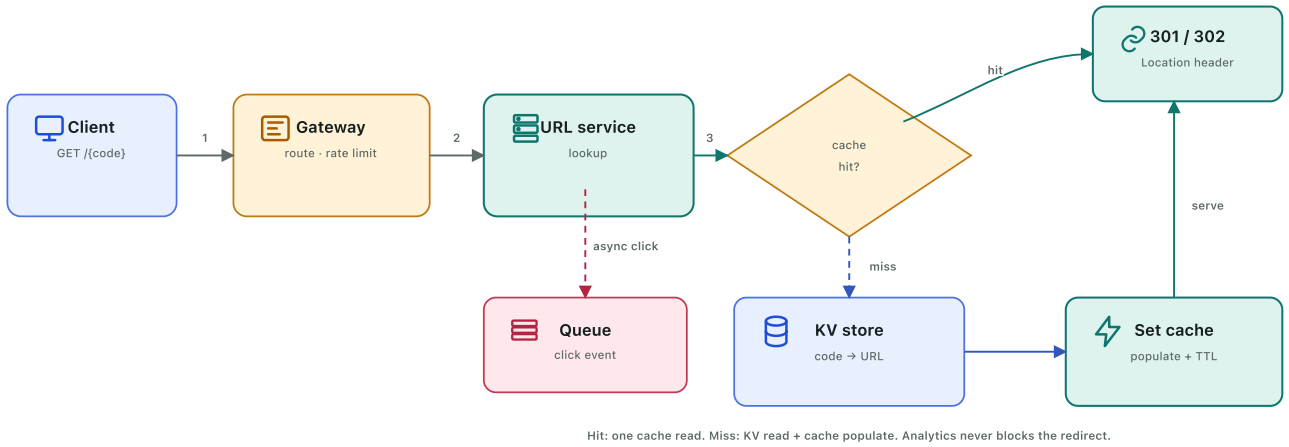


Figure 3 — Redirect flow. The decision is explicit: cache hit or miss; analytics runs separately.

HAPPY PATH

Cache hit

- Gateway routes the request.
- URL service reads the code from cache.
- Return 301/302 immediately.
- Publish click event without waiting.

FALLBACK

Cache miss

- Read the KV store by code.
- Repopulate cache with a TTL.
- Return the redirect; never block on analytics.
- 404 unknown / 410 expired.

HOT-LINK PROTECTION

Cache hot codes, use request coalescing or a short lock against stampedes, add jitter to TTLs, and negative-cache missing codes briefly. A viral link should not turn into thousands of identical KV reads.

8. Optimisation and trade-offs

What to optimise, and in what order

1. **Read path first:** cache hot codes; keep the redirect to one cache lookup.
2. **KV sharding:** shard by hash(code) and replicate each shard across zones.
3. **Write path:** preallocate ID ranges; use conditional writes for uniqueness.
4. **Analytics:** queue/stream clicks; aggregate offline; never block redirects.
5. **Abuse:** rate-limit creates, validate destinations, block malicious domains.

DECISION	GAIN	COST / RISK
Cache-aside	Low redirect latency; less KV load	Staleness; invalidation complexity
301 redirect	Fewer origin requests; browser/CDN caching	Less click visibility; changes can be ignored by clients
302 redirect	Control, expiry, analytics visibility	More traffic reaches the service
Counter + base62	No collisions; short codes	Central ID service or range coordination
Random codes	Hard to guess; no central counter	Collision checks and retries
Async analytics	Redirect stays fast	Eventual counts; queue/worker complexity
Sharding	Scales storage and write throughput	Cross-shard operations are harder
Multi-region reads	Lower latency near users	Replication lag; writes still need a primary

Failure modes

FAILURE	IMPACT	MITIGATION
Cache node down	Higher KV traffic and latency	Replicas, timeouts, KV fallback
Hot KV shard	One shard saturates	Cache hot codes; split/salt; isolate viral links
ID generator down	New links fail	Preallocated ranges; retry; random fallback
Queue backlog	Analytics lag only	Scale workers; sample low-value events
KV failover	Brief read/write errors	Replica promotion; retries with backoff
Expired/malicious link	Bad redirect or abuse	Expiry check; safe-browsing; rate limits

MONITORING

Redirect p99, cache hit ratio, KV latency/errors, hot-shard QPS, ID failures, queue lag, 404/410 rate, and abuse/rate-limit rejections.

9. Follow-ups — one-line answers

Custom alias collision? Use a conditional put on the alias; return 409 instead of overwriting.

Link expiry? Store `expires_at`, check lazily on redirect, return 410, and reclaim space with a cleanup job.

Deleted link still cached? Soft-delete, invalidate the cache, and keep a short negative-cache entry.

Hot/viral link? Cache it, coalesce requests, add a local cache layer, and rate-limit abusive traffic.

Analytics without slowing redirects? Publish a click event to a queue/stream; aggregate asynchronously.

Multi-region? Route users to a nearby read cache; keep writes in a primary region and replicate asynchronously.

Malicious destinations? Validate scheme/domain, use safe-browsing/allowlists, rate-limit creates, and block reported domains.

ID generation down? Preallocate ranges per instance; if exhausted, fall back to random codes with retries.

301 or 302? 301 is cacheable and fewer origin hits; 302 keeps control and analytics visibility.

Cache down? Fall back to the KV store with timeouts and a circuit breaker; serve stale if the product allows it.

Shard key? Hash `code` for even distribution; avoid keys that create one hot shard.

Duplicate create? Use an idempotency key or a unique `(owner_id, long_url)` constraint.

Storage growth? Expire links, archive cold mappings, compress values, and avoid storing media in the KV store.

How many servers? Peak QPS ÷ safe per-server QPS, then add 30–50% headroom.

Traps

AVOID THESE

- Forgetting collision handling.
- Returning a short URL before the mapping is stored.
- Writing analytics synchronously on the redirect path.
- No cache fallback when the cache is down.
- Using a raw auto-increment ID, leaking volume and order.
- Assuming 301 and 302 are interchangeable for analytics.
- Ignoring malicious URLs, expiry, and abuse.
- Sharding before a single KV store is the measured bottleneck.

CLOSING ANSWER

“The design is a read-optimized key-value system: a unique short code maps to a long URL, the redirect path is cache-first with a KV fallback, and analytics is asynchronous. The main trade-offs are 301 vs 302, counter vs random codes, and how much cache staleness the product can tolerate.”